Learning System 3: Logistic Regression and kNN Classification
DT8008 VT26, Halmstad University

Binay Kumar Sah, binsah25@student.hh.se, Master's in Embedded and Intelligence System
Anton Bauer, antbau25@student.hh.se, TAEIS 25

Feburary 2026

Introduction

This lab introduces the concept of classification using logistic regression and kNN methods. Through step-by-step implementations,we built a model to predict university admissions based on exam scores in part A followed by predicting the microchip secletion in part B, respectively. The lab covers data visualization, feature engineering, implementation of the sigmoid function, cost function, and gradient, and optimization using scipy.optimize.minimize. Finally, we evaluated the model's performance and visualize the decision boundary, providing a practical understanding of logistic regression for binary classification tasks.

Part A: Classification with Logistic Regression

Objective

In part A, the task was to build and implement a logistic regression model to solve a classification problem. The goal was to predict the the probability of a student's admission into a university based on their scores from two different exams.By utilizing historical data of previous applicants as a training set, we build a classification model that estimates these admission chances and assigns a class label as 1 for admitted, 0 for not admitted.

Key Concepts

1. **Logistic Regression:** It is a classification method that ensures output stays between 0 and 1, by wrapping the linear equation in a special function called the Sigmoid Function (or Logistic Function).

2. **Sigmoid Function:** A mathematical function used in logistic regression to map any real-valued number into a probability value between 0 and 1. Mathematically, given as :

$$g(z) = \frac{1}{1 + e^{-z}}$$

3. **Binary Regression:** It is the process of predicting one of two possible distinct classes for a given input.

4. **Cost Function(Log Loss):** Unlike linear regression which uses mean squared error, logistic regression uses log loss to penalize incorrect classifications, ensuring the function is convex for optimization. Mathematically, given as :

$$J(\theta) = -\frac{1}{n}\sum_{i=1}^{n}[y^{(i)}\log(h_\theta(x^{(i)})) + (1 - y^{(i)})\log(1 - h_\theta(x^{(i)}))]$$

Algorithm

Listing 1: Logistic Regression Training Algorithm

```
Input:
    X  : Feature matrix (n x d)
    y  : Label vector (n x 1)
Output:
    theta : Optimal parameter vector

Steps:

1. Add bias term to feature matrix:
       X_new = add column of ones to X

2. Initialize parameter vector:
       theta = zeros(d + 1)

3. Define the sigmoid function:
       sigmoid(z) = 1 / (1 + exp(-z))

4. Define hypothesis function:
       h(theta, X) = sigmoid(X * theta)

5. Define cost function:
       E(theta) = -(1/n) * [ y^T log(h) + (1-y)^T log(1-h) ]

6. Define gradient of cost function:
       gradE(theta) = (1/n) * X^T (h - y)

7. Minimize the cost function using an optimization algorithm:
       theta = argmin E(theta)

8. For prediction:
       If h(theta, x) >= 0.5      predict 1
       Else      predict 0

Return optimal theta
```

Observation and Results:

- **Learning Parameters and Decision Boundary:** We implemented the *scipy.optimize.minimize*() function to find the optimal parameters θ for the logistic regression cost function. Figure 1 shows that our initial cost 0.693 reduces to the 0.20 implies that the model successfully learned meaningful parameters. Moreover, the optimal parameters are as $[-25.16, 0.206, 0.201]$, which basically represent the bias term ($\theta_0 = -25.16$), weight for exam $1(\theta_1 = 0.206)$, and $2(\theta_2 = 0.201)$, respectively. Since θ_1 and θ_2 are positive, both indicate a higher probability of admission and contribute equally to the decision.

```
Initial cost:  0.6931471805599452
Best Parameter = [-25.16131857   0.20623159   0.20147149]
Optimal Cost = 0.20349770158947486
```

Figure 1: Output of optimal parameters and reduced cost

Similarly, figure 2 shows the decision boundary (green line) separates admitted(blue stars) and non-admitted(red circles). Most data points are correctly separated. Hence this shows that our classifier creates a clear line of separation with high accuracy.

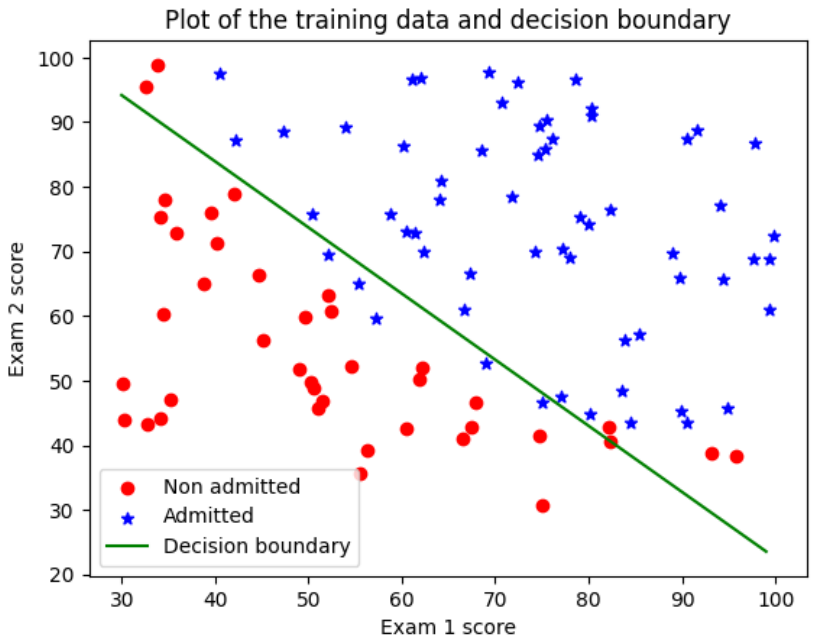


Figure 2: Plotting of decision boundary

- **Evaluation:** After learning the parameters based on the student records, we utilize the model to predict the data of student having 45 score in exam 1 and 85 in exam 2, our model is successfully predict the possibility of admission as 77.62%. To evaluate a prediction to the give data set and then comparing it to the actual result, accuracy was found to be of 89% as shown in figure 3. Thus, this shows that our model can classify the students possibility of admission with high accuracy.

```
y pred = [0 0 0 1 1 0 1 0 1 1 1 0 1 1 0 1 0 0 1 1 0 1 0 0 1 1 1 1 1 0 0 1 1 0 0 0 0 1
 1 0 0 1 0 1 1 0 0 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 0 0 0 0 0 1 0 1 1 0 1 1 1
 1 1 1 1 0 1 1 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 1 1 1 1 0 1]
Accuracy :  89.00%
```

Figure 3: Output of data evaluation

Part B: Classification with kNN

Objective

In Part B, the task was to implement kNN on a dataset from a microchips fabrication plant for quality assurance. The dataset contained the results of two microchip tests and if those resulted in the microchip being rejected or accepted. Furthermore, data visualization was done to justify/discuss why kNN is a suitable algorithm for the problem at hand.

Key Concepts

- **K-Nearest Neighbours:** Is an algorithm that takes into consideration all the training data points X when trying to predict the value of a new data point x, as opposed to Logistic Regression, that uses the training data points to learn the parameters θ best describing the decision boundary. In kNN, the training data points are used to calculate the distances to the given data point x using the Euclidean distance. A hyperparameter k is then used to vary the scope of consideration for assigning the prediction y to x, that is, the k-number of closest neighbors that are able to influence the prediction of the new point. The classification is then created by taking the majority vote among the neighbors.

- **Euclidean Distance:** Is the straight line distance between two points in Euclidean Space, and is calculated via the Pythagorean theorem. Given that vectors can be thought of as points in space, the Euclidean distance for vector u and v is the following:

$$\|u - v\| = \sqrt{\sum_{j=1}^{d} (u_j - v_j)^2},$$

  where $\|u - v\|$ is the norm (L2), or size of the distance vector between u and v, that is the same as taking the Euclidean distance (to the right of the equal sign).

Algorithm

1. Load the data containing the microchips's test data.
2. Calculate the distance for every data point in the training data to the new test data point/points.
3. Sort the calculated distances and save the k-first closest data points (neighbors) for each new point.
4. Determine the classification of each new point by taking the majority vote of the saved closest neighbors's values.
5. Display the results either through a console print, such as accuracy of predictions (training vs test), or through a plot.

Observation and Results

- **kNN Classification**: The first notable observation was that of the distribution of the data, seen in Figure 4, where it is apparent that a straight line cannot separate the data into accurate classifications. As such, applying logistic regression without feature engineering would yield sub-optimal results. Hence the application of kNN instead, as it is a nonlinear classifier.

  Further observations surfaced when plotting the decision boundary for various values of k (1,15,30) as shown in Figure 5. Figure 5a displays what happens when k is small, where jagged edges and isolated points can be seen. This is a sign of overfitting as the model has reacted strongly to each individual data point; i.e. captured the noise rather than the underlying pattern. A k of

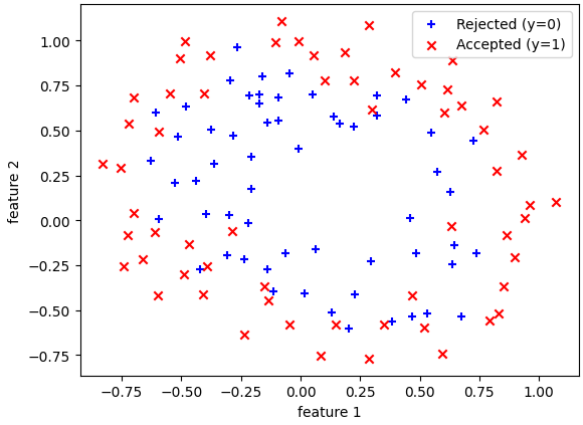
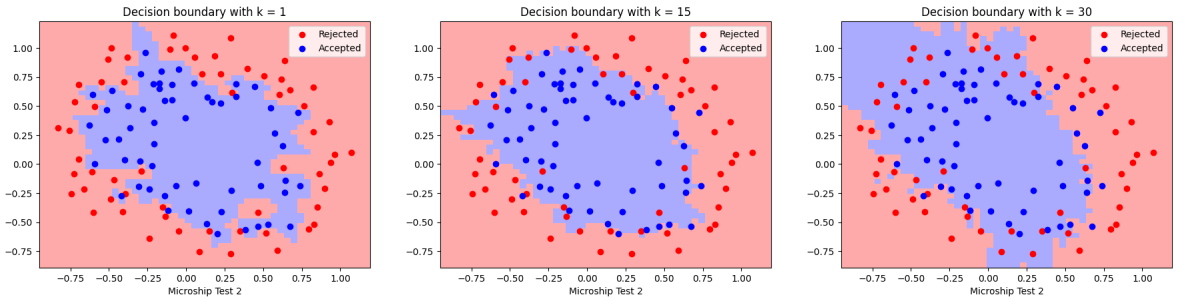


Figure 4: Scatter plot of the features (microchip tests) and corresponding label

15, seen in Figure 5b, gives a smoother edge and no longer isolates outliers, providing a balance between bias and variance. Too high of a k, seen in Figure 5c with a k of 30, fails to capture the correct decision boundary as too much of an emphasis is put on the many neighbors - making it unable to capture the more complex geometry needed. Therefore, the most suitable model is the one with a k of 15 as it seems to correct capture the underlying pattern, along with a training accuracy of about 80.5%.



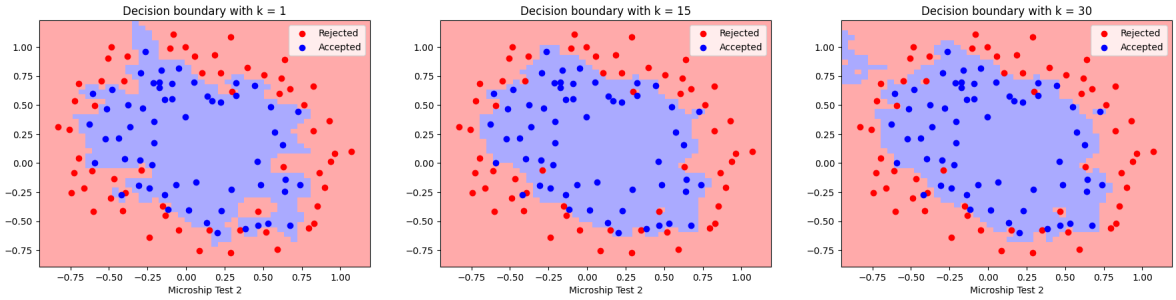
(a) Plot of kNN decision boundary when $k = 1$

(b) Plot of kNN decision boundary when $k = 15$

(c) Plot of kNN decision boundary when $k = 30$

Figure 5: Plots of weighted kNN decision boundaries with varying k-values

- **Weighted kNN classification**: Unlike to kNN, where every neighbor is treated equally, weighted kNN neighbors are treated on weight basis. Closer neighbors have higher weight and is calculated by the [1/distance]. Figure 6 shows the weighted kNN decision boundary with k equal to 1, 15, and 30, respectively. Here, we can observe that the modal prioritizes the local information, and unlike kNN, even at high k, the weighted kNN decision boundary remains more refined. This provides more robust classification by reducing the bias introduced by distant neighbors in a large k neighborhood. It effectively balances the smoothing effect of a large k with the accuracy of local data.



(a) Plot of weighted kNN decision boundary when $k = 1$

(b) Plot of weighted kNN decision boundary when $k = 15$

(c) Plot of weighted kNN decision boundary when $k = 30$

Figure 6: Plots of weighted kNN descision boundaries with varying k-values

Conclusion

Lab 3 taught us that Logistic Regression is a useful classification tool when used on suitable data, such as the school admission, where a straight line can be used to create an effective decision boundary. The opposite can also be said for kNN, where at the cost of being computationally heavy, it can account for a more complex decision boundary some data might need, as seen with the microchip quality assurance. Ultimately this highlights that the correct algorithm for the job is highly dependent on the data we're working with, in turn, pointing at usefulness of techniques such as visualization.